

External vs Internal Services

1. Overview

In system design, engineers often secure **external/public-facing services** but neglect **internal services**.

That is risky.

Modern design principle:

Internal does not automatically mean trusted.

Every service—external or internal—should be evaluated for:

- Security
- Privacy
- Access control
- Auditing
- Abuse prevention

2. Definitions

A. External Services

Services directly reachable by users, partners, or internet traffic.

Examples:

- Public APIs
- Mobile backend
- Web frontend APIs
- Partner integration APIs

B. Internal Services

Services used only inside organization/network.

Examples:

- Rate limiter service
- User profile service
- Payment orchestration service
- Analytics pipeline
- Notification service
- Internal admin APIs

3. Old Assumption (Unsafe)

Traditional thinking:

None

Outside network = dangerous

Inside network = trusted

This model is outdated.

Reasons:

- Insider threats
- Misconfigured services
- Stolen credentials
- Compromised internal host
- Lateral movement after breach
- Human mistakes

4. Modern Principle: Zero Trust

Trust nothing automatically.

Verify every request:

None

Who are you?

What can you access?

Why do you need it?

Applies internally too.

5. Why Internal Services Need Security

Even internal rate limiter or config services may expose:

- Traffic patterns
- User IDs
- Abuse signals
- Infrastructure topology
- Sensitive controls

Compromise can cascade.

6. Example Risk: Internal Service Without Auth

Suppose internal service:

None

GET /all-users

No auth because “internal only”.

If one internal host compromised, attacker can query everything.

7. Internal Threat Sources

A. Compromised Service

One vulnerable service attacks others.

B. Insider Abuse

Employee misuses access.

C. Misconfiguration

Wrong firewall/security group.

D. Credential Leakage

API keys exposed in logs/code.

8. Security by Default

Best engineering culture:

Every service starts secure, then selectively opens minimal access.

Not the reverse.

9. Internal Authentication

Use machine/service identity.

Examples:

- mTLS certificates
- Service tokens
- Workload identity
- Signed JWTs

Service A proves identity to Service B.

10. Internal Authorization

Even authenticated services need permissions.

Example:

None

Order service can read user shipping address

Analytics service cannot

Least privilege.

11. Role-Based Access Control (RBAC)

Use RBAC internally for:

- Git repos
- CI/CD pipelines
- Dashboards
- Admin panels
- Production data tools

Most mature organizations already do this.

12. Just-in-Time Access

Temporary elevated access only when needed.

Example:

None

DB admin access for 1 hour incident

Then auto revoked

Reduces standing privilege risk.

13. Privacy for Internal Services

Internal services should avoid unnecessary user data replication.

Bad:

None

Every service stores full user profile

Good:

None

Most services store user_id only

Sensitive data stays in user-profile service

14. Data Minimization Internally

Only services needing PII should access PII.

Others use:

- User IDs
- Tokens
- References
- Aggregated data

15. User ID Indirection

Instead of spreading personal data:

None

`service stores user_id`

`fetches profile only when needed`

This limits privacy blast radius.

16. Rotatable User Identifiers

In some architectures, internal IDs can be rotated after breach/privacy incident.

Helps reduce long-term linkage.

17. Sensitive Databases Behind Services

Best practice:

None

Do not allow many apps direct DB access

Instead:

None

Access via dedicated service layer

Benefits:

- Centralized auth
- Auditing
- Validation
- Policy enforcement

18. Logging and Auditing

All sensitive internal access should be logged:

- Who accessed
- Which service
- Which user data
- Time
- Action taken

Needed for incident response.

19. Example: Customer Support Tool

Support team may need customer info.

Use:

- Masked fields
- Ticket-based access
- Session logging
- Temporary access grants

20. Service Mesh Security

A **service mesh** can enforce internal policies.

Examples include:

- mTLS
- Traffic policy
- Identity
- Authorization
- Observability

Popular tools:

- Istio
- Linkerd

21. External vs Internal Security Comparison

Topic	External Services	Internal Services
Authentication	Mandatory	Mandatory
Authorization	Mandatory	Mandatory
TLS	Yes	Prefer yes / mTLS
Rate limiting	Yes	Sometimes yes
Audit logs	Yes	Yes

Privacy controls	Yes	Yes
------------------	-----	-----

22. Common Mistakes

“Internal means safe”

Shared admin credentials

Direct DB access by many apps

No audit logs internally

Broad employee permissions

PII copied into many systems

23. Interview Answer Example

Q: Do internal microservices need auth?

Answer:

Yes. I'd avoid trusting network location alone. Internal services should authenticate using service identity, authorize via least privilege policies, encrypt traffic where appropriate, and log access to sensitive data.

24. Best Practices

- Adopt zero trust mindset
- Use mTLS for service-to-service traffic
- Centralize IAM
- Restrict DB access behind APIs
- Minimize PII copies
- Temporary privilege elevation
- Audit everything sensitive
- Rotate credentials regularly

25. Design Heuristic

Ask:

None

If one internal host is compromised,
what else can attacker reach?

That reveals missing controls quickly.

26. Short Revision Notes

None

Internal $\neq$ Trusted

Secure both external and internal services:

- AuthN
- AuthZ
- Least privilege
- Logging
- Privacy controls
- Minimal PII spread